UNIT 1

INTRODUCTION TO DATA STRUCTURES

Syllabus

1.1 Introduction

- Concept of Data Structure
- Need for Data Structures
- Definition of Data Structure
- Abstract Data Type (ADT)

1.2 Types of Data Structures

- Linear Data Structures
- Non-Linear Data Structures

1.3 Operations on Data Structures

- Traversing
- Insertion
- Deletion

1.1 INTRODUCTION TO DATA STRUCTURES

1.1.1 What is Data?

Data refers to a collection of raw facts, figures, values, or information that can be processed by a computer.

Examples:

- Student name: `Rahul`
- Student age: `22`
- Marks: `85`

Data by itself may not provide meaningful information until it is processed.

Example

Suppose we have the following marks:

```
75, 82, 65, 90, 88
```

These are raw data values.

After processing:

- Total = 400
- Average = 80
- Highest = 90
- Lowest = 65

The processed data provides useful information.

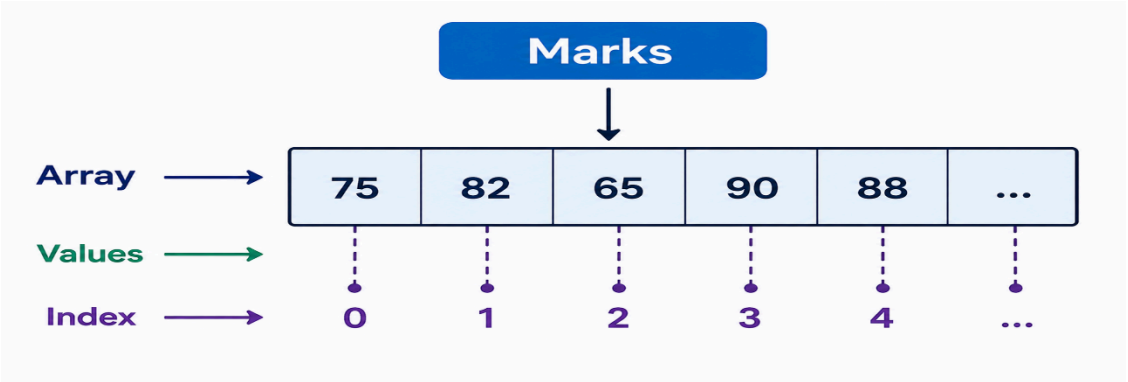
1.1.2 What is a Data Structure?

A **Data Structure** is a systematic way of organizing, storing, and managing data in a computer so that it can be accessed and processed efficiently.

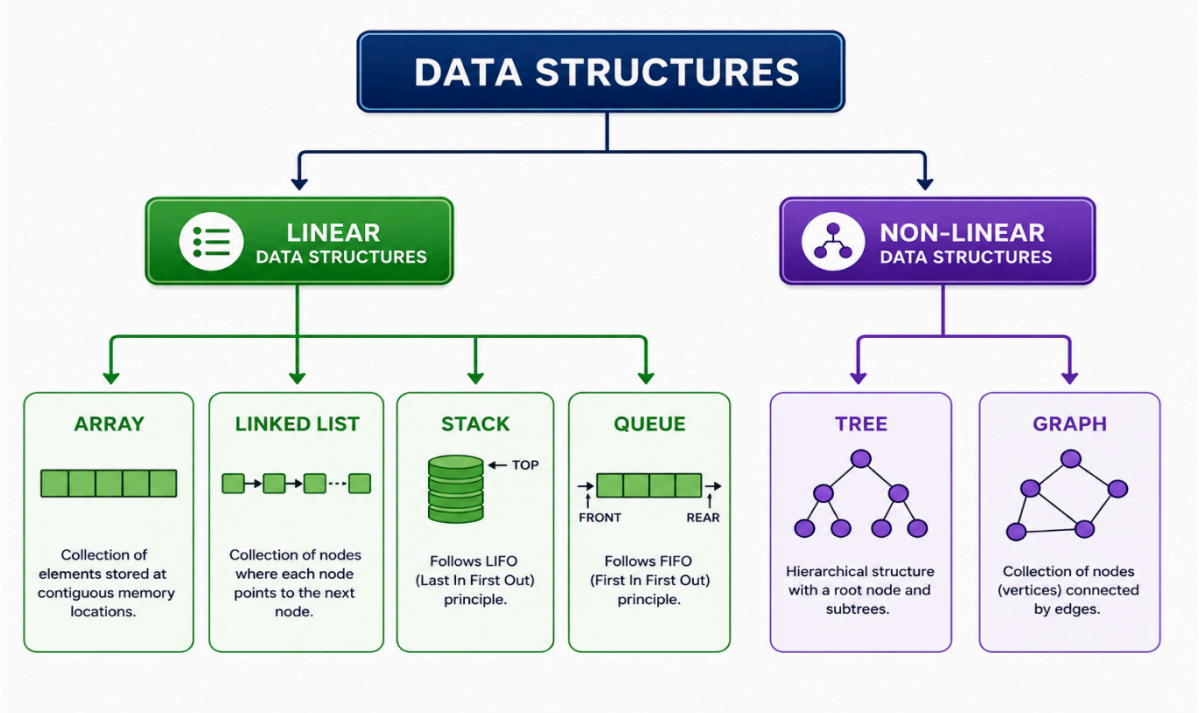
Example

Suppose we need to store marks of 100 students.

We can store them using an array:



Here, the array provides an organized structure for storing the marks.



1.1.3 Concept of Data Structure

The concept of a data structure involves three major aspects:

1. Organization of Data

Data must be organized in a meaningful way.

For example:

```
Student Marks
    Student 1 → 80
    Student 2 → 75
    Student 3 → 91
    Student 4 → 68
```

2. Storage of Data

The structure determines how data is stored in memory.

For example:

- Array → Contiguous memory
- Linked List → Nodes connected through pointers
- Tree → Hierarchical organization
- Graph → Vertices and edges

3. Processing of Data

The data structure should allow operations such as:

- Searching
- Sorting
- Insertion
- Deletion
- Traversal
- Updating

1.1.4 Why Do We Need Data Structures?

As the amount of data increases, managing data efficiently becomes difficult.

Consider a university with:

- 10,000 students
- 500 faculty members
- 50,000 examination records
- Thousands of courses

If the data is not properly organized, performing operations on it becomes slow and complicated.

Data structures help us manage large amounts of data efficiently.

Major Reasons for Using Data Structures

1. Efficient Data Organization

Data structures organize data systematically.

Example:

An array can store student marks:

```
[78, 85, 92, 67, 74]
```

Instead of storing every value separately.

2. Efficient Searching

Data structures can make searching faster.

For example:

- Linear Search → `O(n)`
- Binary Search → `O(log n)`

A suitable data structure can significantly improve search performance.

3. Efficient Insertion and Deletion

Different data structures provide different efficiencies for insertion and deletion.

For example:

A linked list can insert a new node without shifting all existing elements.

4. Efficient Memory Utilization

Data structures help utilize memory effectively.

For example:

A linked list can dynamically allocate memory as required.

5. Handling Large Amounts of Data

Modern applications generate huge amounts of data.

Examples:

- Social media
- Banking systems
- E-commerce
- Cloud computing
- Big Data systems
- Artificial Intelligence
- Database systems

Data structures help manage this data efficiently.

6. Improving Program Performance

The choice of data structure directly affects program performance.

For example:

If we frequently need to search data, a suitable structure such as a hash table can provide very fast searching

7. Reusability

Well-designed data structures can be reused in different programs.

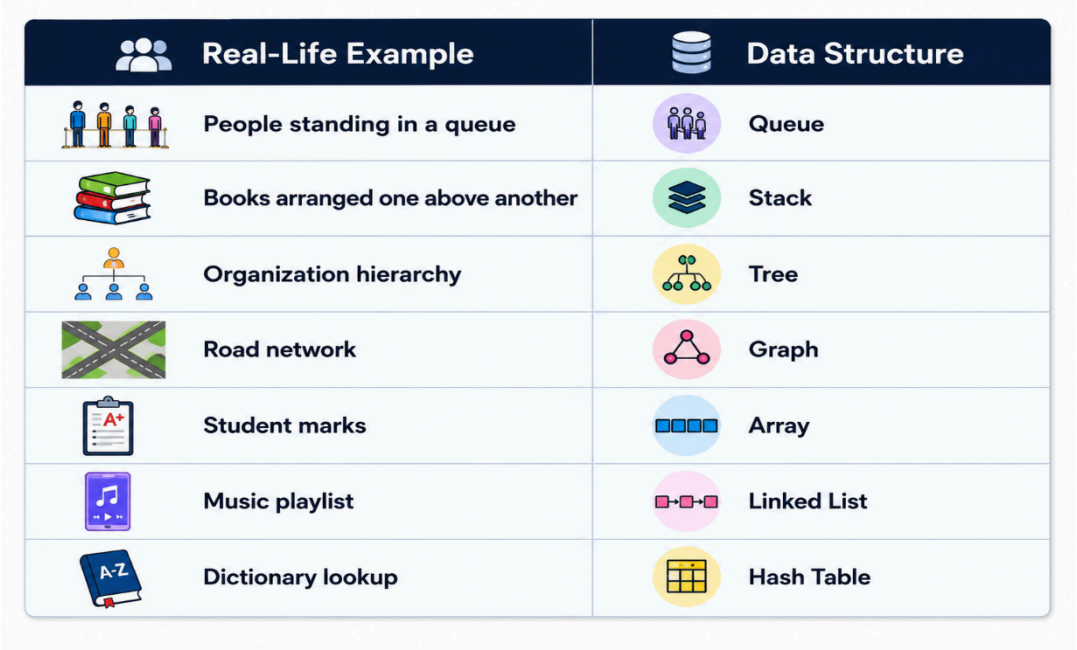
For example:

A stack can be used in:

- Function calls
- Expression evaluation
- Undo operations
- Browser history

1.1.5 Real-Life Examples of Data Structures

Data structures are not limited to programming. Many real-life systems follow similar organizational concepts.



1.1.6 Characteristics of a Good Data Structure

A good data structure should provide:

1. Efficient storage
2. Efficient access

3. Efficient insertion
4. Efficient deletion
5. Efficient searching
6. Appropriate memory utilization
7. Easy implementation
8. Scalability

1.1.7 Data Structure and Algorithm

Data structures and algorithms are closely related.

Data Structure

Defines how data is organized and stored.

Algorithm

Defines how a problem is solved using that data. Step by step approach to solve particular problem.

For example:

Suppose student marks are stored in an array.

```
Array
[75, 89, 65, 92, 78]
```

An algorithm can be used to find the highest marks.

Data Structure + Algorithm = Efficient Solution

1.1.8 Abstract Data Type (ADT)

What is an Abstract Data Type?

An **Abstract Data Type (ADT)** is a logical description of a data type that specifies:

1. What data is stored

2. What operations can be performed

without specifying how those operations are implemented internally.

Simple Definition

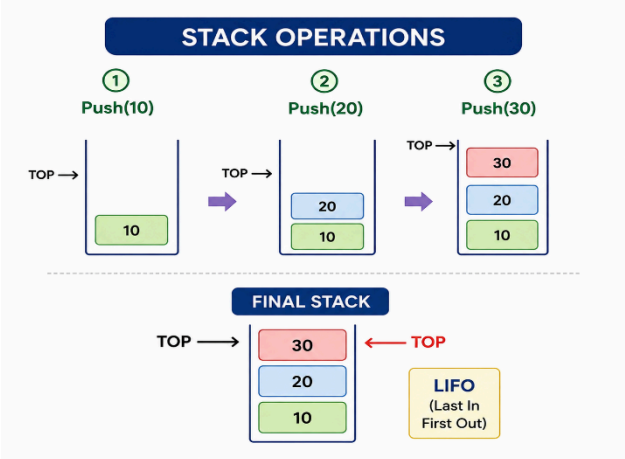
An Abstract Data Type is a mathematical/logical model of a data type that defines its data and operations independently of implementation.

Example: Stack ADT

A stack follows the **LIFO – Last In, First Out** principle.

Suppose:

```
Push(10)
Push(20)
Push(30)
```



Possible stack operations:

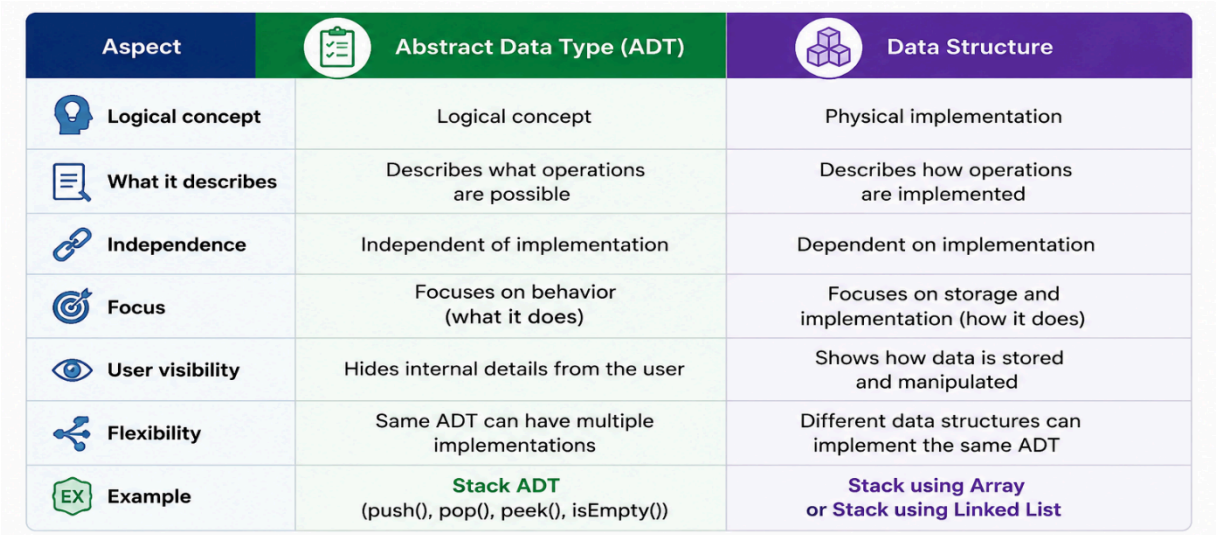
- `push()`
- `pop()`
- `peek()`
- `IsEmpty()`

The ADT specifies what these operations do.

It does not necessarily specify whether the stack is implemented using:

- Array
- Linked List

1.1.11 ADT vs Data Structure



Aspect	Abstract Data Type (ADT)	Data Structure
Logical concept	Logical concept	Physical implementation
What it describes	Describes what operations are possible	Describes how operations are implemented
Independence	Independent of implementation	Dependent on implementation
Focus	Focuses on behavior (what it does)	Focuses on storage and implementation (how it does)
User visibility	Hides internal details from the user	Shows how data is stored and manipulated
Flexibility	Same ADT can have multiple implementations	Different data structures can implement the same ADT
Example	Stack ADT (push(), pop(), peek(), isEmpty())	Stack using Array or Stack using Linked List

1.2.1 Linear Data Structures

A **linear data structure** is a data structure in which elements are arranged sequentially or linearly.Each element generally has a unique predecessor and successor, except the first and last elements.

Examples

- Array
- Linked List
- Stack
- Queue

Characteristics of Linear Data Structures

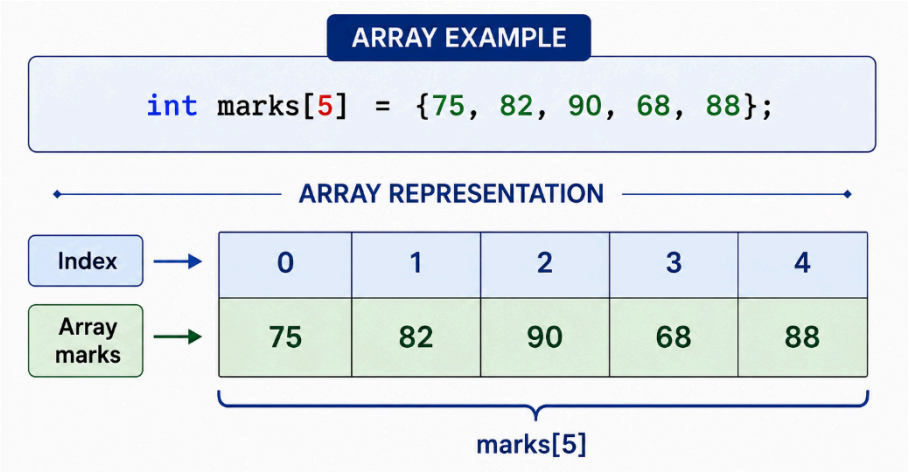
1. Elements are arranged sequentially.
2. Traversal generally occurs in a linear manner.
3. Every element has a logical relationship with the previous/next element.
4. Implementation can be sequential or linked.
5. Suitable for sequential processing.

1.2.2 Array

An **array** is a collection of elements of the same data type stored in contiguous memory locations.

Example:

```
int marks[5] = {75, 82, 90, 68, 88};
```



Advantages

- Fast random access
- Simple implementation
- Efficient memory access
- Easy traversal

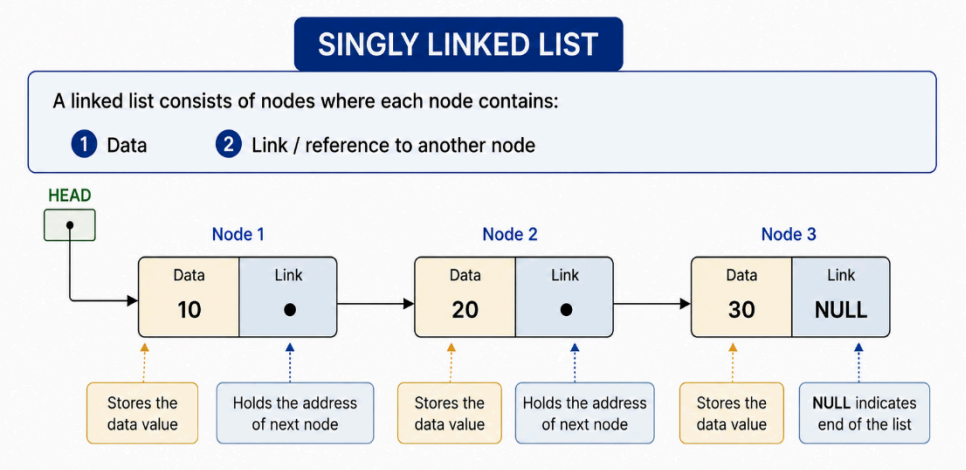
Disadvantages

- Fixed size in traditional arrays
- Insertion can be expensive
- Deletion can be expensive
- Requires contiguous memory

1.2.3 Linked List

A linked list consists of nodes where each node contains:

1. Data
2. Link/reference to another node



Advantages

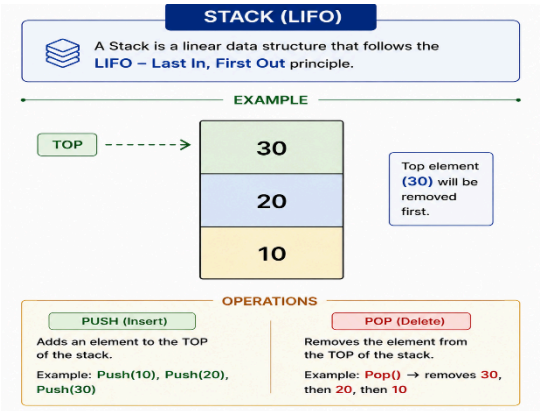
- Dynamic size
- Easy insertion and deletion
- Does not require contiguous memory

Disadvantages

- Extra memory required for links
- No direct/random access
- Traversal is generally sequential

1.2.4 Stack

A **stack** is a linear data structure that follows the:



If `30` was inserted last, it will be removed first.

Main Operations

- Push
- Pop
- Peek/Top

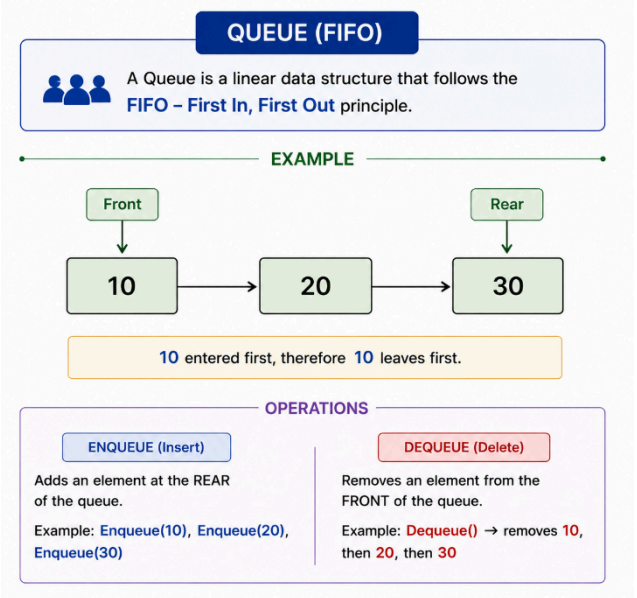
Applications

- Function calls
- Recursion
- Undo/Redo
- Expression evaluation
- Parentheses matching
- Browser history

1.2.5 Queue

A **queue** is a linear data structure that follows:

FIFO – First In, First Out.



10 entered first, therefore 10 leaves first.

Main Operations

- Enqueue
- Dequeue
- Front/Peek

Applications

- CPU scheduling
- Printer queue
- Network buffering
- Customer service systems
- Task scheduling

1.2.6 Non-Linear Data Structures

A **non-linear data structure** is a data structure in which elements are not arranged sequentially.

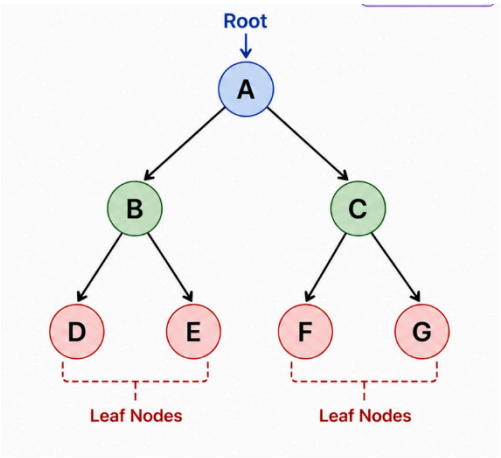
Elements may have hierarchical or network-based relationships.

Examples

- Tree
- Graph

1.2.7 Tree

A **tree** is a hierarchical non-linear data structure consisting of nodes connected through edges.



Here:

- A → Root
- B, C → Children of A
- D, E → Children of B & Leaf Nodes
- F, G → Children of C & Leaf Nodes

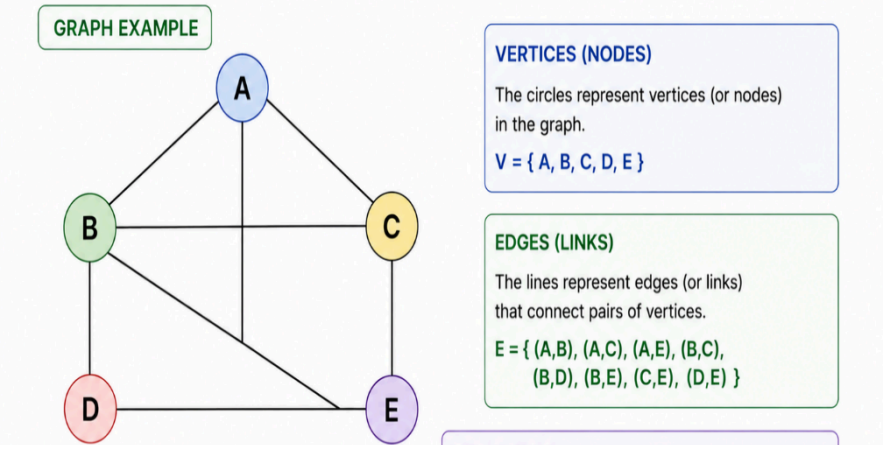
Applications

- File systems
- Database indexing
- XML/HTML document structures
- Organization hierarchy
- Artificial intelligence
- Decision trees

1.2.8 Graph

A **graph** is a non-linear data structure consisting of:

- Vertices (nodes)
- Edges (connections)



Applications:

- Social networks
- Google Maps
- Computer networks
- Transportation systems
- Recommendation systems
- Network routing

1.2.9 Linear vs Non-Linear Data Structures

LINEAR DATA STRUCTURE	NON-LINEAR DATA STRUCTURE
Sequential	Hierarchical / Network
One-to-one generally	One-to-many / Many-to-many
Usually sequential	Multiple possible paths
Array, Stack, Queue, Linked List	Tree, Graph
Generally simpler	Generally more complex
Sequential processing	Hierarchical / Network relationships

1.3 OPERATIONS ON DATA STRUCTURES

A data structure becomes useful because different operations can be performed on the stored data.

Common operations include:

1. Traversing
2. Insertion
3. Deletion
4. Searching
5. Sorting
6. Updating
7. Merging

In this unit, we focus on:

- Traversing
- Insertion
- Deletion

1.3.1 Traversing

Traversal is the process of visiting or accessing every element of a data structure exactly once, usually for processing or displaying the elements.

Example

Consider:

```
Array = [10, 20, 30, 40, 50]
```

Traversal means visiting:

```
10 → 20 → 30 → 40 → 50
```

Array Traversal

In C:

```
for(int i = 0; i < n; i++)
{
    printf("%d ", arr[i]);
}
```

For an array of `n` elements, all elements are visited.

Time Complexity

O(n)

1.3.2 Insertion

Definition

Insertion is the process of adding a new element or node to a data structure.

Example

Initial array:

```
10 20 30 40
```

Insert `25` between 20 and 30:

```
10 20 25 30 40
```

Insertion in an Array

When inserting an element into the middle of an array, existing elements may need to be shifted.

Example:

```
Before:

10 20 30 40
      ↑
    Insert 25
```

Elements are shifted:

```
10 20 30 40
      ↓  ↓
10 20 25 30 40
```

1.3.3 Deletion

Deletion is the process of removing an existing element or node from a data structure.

Deletion from Array

When an element is deleted from the middle, subsequent elements may need to be shifted.

Example:

```
Before:

10 20 30 40 50
      ↑
    Delete
```

After shifting:

```
10 20 40 50
```

Therefore, deletion from the beginning or middle generally requires shifting.

1.3.7 Real-World Applications

Arrays

Used in:

- Student marks
- Matrices
- Image processing
- Tables
- Numerical computation

Linked Lists

Used in:

- Dynamic memory management
- Music playlists
- Browser navigation

- Implementing stacks and queues

Stacks

Used in:

- Undo/Redo
- Function calls
- Recursion
- Expression evaluation
- Syntax parsing

Queues

Used in:

- CPU scheduling
- Printer scheduling
- Customer service systems
- Network packet processing

Trees

Used in:

- File systems
- Databases
- Compilers
- Decision-making systems

Graphs

Used in:

- Social networks
- Maps
- Network routing
- Recommendation systems

Questions

Q.1 Define/One line/List (1 Mark)

1) Define Data Structure.
2) Define Abstract Data Type (ADT).
3) List the examples of Linear Data Structures and Non-Linear Data Structures.
4) Define Traversing, Insertion, and Deletion.

Q.2 Explain (2-3 Marks)

1) Explain the concept of Data Structure with a suitable example.
2) Explain the need for Data Structures and discuss any four reasons for using them.
3) Explain Abstract Data Type (ADT) with the example of Stack ADT.
4) Explain Linear and Non-Linear Data Structures with suitable examples.
5) Explain Graph and Tree Data Structures in short.

Q.3 Apply (3-4 Marks)

1) Given the array `[10, 20, 30, 40, 50]`, demonstrate the traversal operation and state its time complexity.
2) Demonstrate the insertion of `25` between `20` and `30` in the array `[10, 20, 30, 40]`. Show the array after insertion.
3) Given the operations `Push(10)`, `Push(20)`, and `Push(30)`, demonstrate the resulting Stack and identify the element removed by `Pop()`.

Q.4 Analyse Differentiate (5 Marks)

1) Differentiate between Abstract Data Type (ADT) and Data Structure based on their characteristics and implementation.
2) Differentiate between Linear and Non-Linear Data Structures with suitable examples**.**

Q.5 (5 Marks)

A student marks management system requires storing and processing marks of a large number of students. Evaluate the suitability of an Array or Linked List for this application, select the appropriate Data Structure, and write a C program to demonstrate its use. Justify your choice.